

PROGRAMACIÓN MULTIMEDIA Y DISPOSITIVOS MÓVILES

TEMA 10:

ALMACENAMIENTO PERSISTENTE DE DATOS



ÍNDICE

1. La clase SharedPreferences.
2. SQLite con Room
3. Conexión Firebase



- En este curso, vamos a ver 2 alternativas que nos ofrece Android para el almacenamiento permanente de datos en local (es decir, que estos no se borran cuando se cierra la aplicación o apagamos el dispositivo).
- Según el tipo de necesidades que tengamos, podemos usar alguno de estos dos métodos:
 - La clase SharedPreferences (archivo XML).
 - Una base de datos con acceso SQL.
- Además, una aplicación puede utilizar perfectamente ambos métodos para realizar el almacenamiento de datos.

PMDM

ALMACENAMIENTO PERSISTENTE DE
DATOS



1. Clase SharedPreferences

1.- Clase SharedPreferences

- Cuando tenemos que almacenar una pequeña cantidad limitada de datos es adecuado utilizar la clase *SharedPreferences*. Por ejemplo, para configuraciones de la aplicación como pueden ser colores de pantalla, el nivel actual en un juego, las opciones elegidas en la app...
- Para probar el funcionamiento de este tipo de almacenamiento vamos a realizar la siguiente aplicación:

1.- Clase SharedPreferences

- Actividad que solicite el email de una persona al usuario. La aplicación debe cumplir los siguientes requisitos:
 - Guardar el email ingresado utilizando la clase SharedPreferences.
 - Disponer un botón para almacenar el email ingresado.
 - Cada vez que se inicie la aplicación, cargar en el EditText el último email ingresado.
- Quedaría de la siguiente manera:



1.- Clase SharedPreferences

- **PARA GUARDAR EL DATO**, lo primero que se hace es crear un objeto de la clase SharedPreferences:
val preferencias: SharedPreferences = getSharedPreferences ("datos", Context.MODE_PRIVATE)
- "datos" será el nombre del archivo, y la forma de creación del archivo "MODE_PRIVATE" indica que solo esta aplicación puede consultar el archivo XML que se crea.
- Si necesitas que tu app comparta este archivo con otras apps deberás utilizar el componente *FileProvider*. Para más información, visita:

<https://developer.android.com/training/secure-file-sharing?hl=es-419>

1.- Clase SharedPreferences

- Después creamos un objeto de la clase Editor y obtenemos la referencia del objeto de la clase SharedPreferences que acabamos de crear. Mediante el método `putString` almacenamos en la variable "email" el valor del String cargado en el EditText:

```
val editor: SharedPreferences.Editor = preferencias.edit()
```

```
editor.putString("email", editTextEmail.text.toString())
```

- Cuando guardamos datos en el archivo de preferencias, podemos almacenar distintos tipos de datos según el método que llamemos en el momento de grabar:
 - `editor.putInt("edad",3);`
 - `editor.putBoolean("activo", true);`
 - `editor.putFloat("altura", 2.3f);`

1.- Clase SharedPreferences

- Por último debemos llamar al método commit de la clase Editor para que el dato quede almacenado de forma permanente en el archivo de preferencias. Esto hace que cuando volvamos a arrancar la aplicación se recupere el último email ingresado.

editor.commit();

1.- Clase SharedPreferences

- **PARA RECUPERAR EL DATO**, y que aparezca en el EditText cuando abramos nuestra aplicación, crearemos en el método onCreate un objeto de la clase SharedPreferences, recuperando a continuación el dato y asignándoselo al EditText:

```
val preferencias: SharedPreferences=getSharedPreferences("datos", Context.MODE_PRIVATE)  
  
val email: String? = preferencias.getString("email", "")  
  
if(email != null) {  
  
    editTextEmail.setText(email)  
  
}
```

- Para extraer los datos del archivo de preferencias debemos indicar el nombre a extraer y un valor de retorno si dicho nombre no existe en el archivo. En nuestro ejemplo, la primera vez que se ejecute nuestro programa, como es lógico, no existe el archivo de preferencias, lo que hace que Android cree un archivo internamente, pero el método "getSharedPreferences" nunca devolverá un objeto nulo. Por tanto, si tratamos de extraer el valor de "email" se retornará el segundo parámetro (es decir, el String con una cadena vacía ""), pero nunca un valor null.

EJERCICIOS

- **Ejercicio 01:** Crea un proyecto en Android Studio llamado "Mini Agenda" que contenga una Actividad.
- Este proyecto consistirá en crear una pequeña agenda mediante la clase SharedPreferences.

EJERCICIOS

- Quedaría de la siguiente manera:



5:58

Nombre:

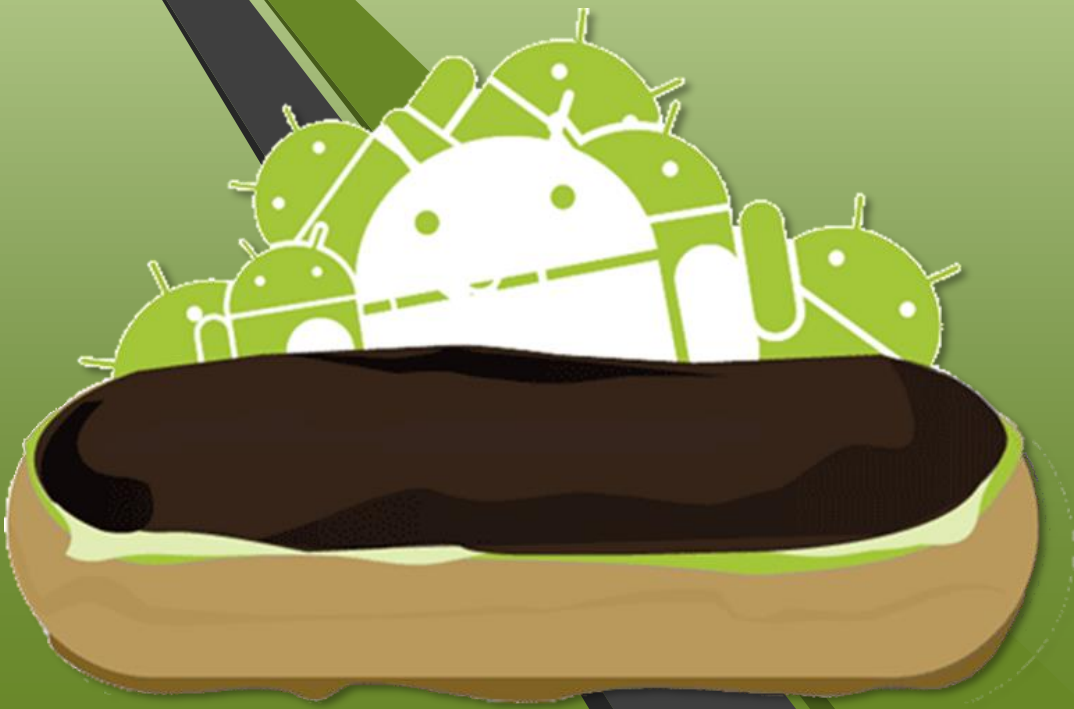
Teléfono:

GUARDAR BUSCAR

PMDM

ALMACENAMIENTO PERSISTENTE DE
DATOS

2. SQLite con Room



2.- SQLite con Room

❖ ¿Qué es SQLite?

- SQLite es una base de datos relacional y Open Source, integrada en Android, que permite el almacenamiento persistente de datos en los dispositivos.
- Las principal ventaja de usar SQLite es que no requiere configuración ni tiene un servidor de base de datos ejecutándose en un proceso separado. Además, su empleo es relativamente simple.

2.- SQLite con Room

- Para utilizar directamente SQLite, Android incluye las clases SQLiteDatabase, SQLiteOpenHelper y Cursor de forma nativa.
- El problema es que trabajar con SQLite "a bajo nivel" presenta muchas desventajas, como la generación de mucho código repetitivo, implementación de consultas SQL directamente en Strings o apps poco seguras y difíciles de mantener.

2.- SQLite con Room

❖ ¿Qué es Room?

- Room es una biblioteca de persistencia de datos que ofrece una capa de abstracción sobre SQLite, facilitando el trabajo mediante objetos y anotaciones. No sustituye a SQLite, sino que utiliza la base de datos internamente.
- Entre las principales ventajas de Room, podemos destacar que evita la implementación de código repetitivo, aumenta la seguridad y la legibilidad del mismo y es compatible con las Corrutinas de Kotlin.

2.- SQLite con Room

- Para ver el funcionamiento de SQLite con Room vamos a crear una pequeña base de datos de una tienda que guarde su stock de artículos en una tabla, que contendrá los siguientes campos: código, descripción, stock y precio.
- La aplicación constará de una única actividad que debe permitir:
 1. Añadir artículos.
 2. Consultar artículos por el código.
 3. Consultar artículos por la descripción.
 4. Borrar artículos.
 5. Modificar artículos.

2.- SQLite con Room

- La interfaz de nuestra app será sencilla y quedaría de la siguiente manera:



2.- SQLite con Room

Configuración previa

- Para poder utilizar SQLite con Room en nuestra aplicación móvil, debemos añadir lo siguiente en el fichero *build.gradle.kts* de *Module:app* y sincronizar:
- Plugins:

```
plugins {  
    alias(libs.plugins.android.application)  
    alias(libs.plugins.kotlin.android)  
  
    //Plugin que permite procesar las anotaciones que utilizamos con Room,  
    //leer las anotaciones y generar las clases automáticas para la BD y el DAO  
    kotlin("kapt")  
}
```

2.- SQLite con Room

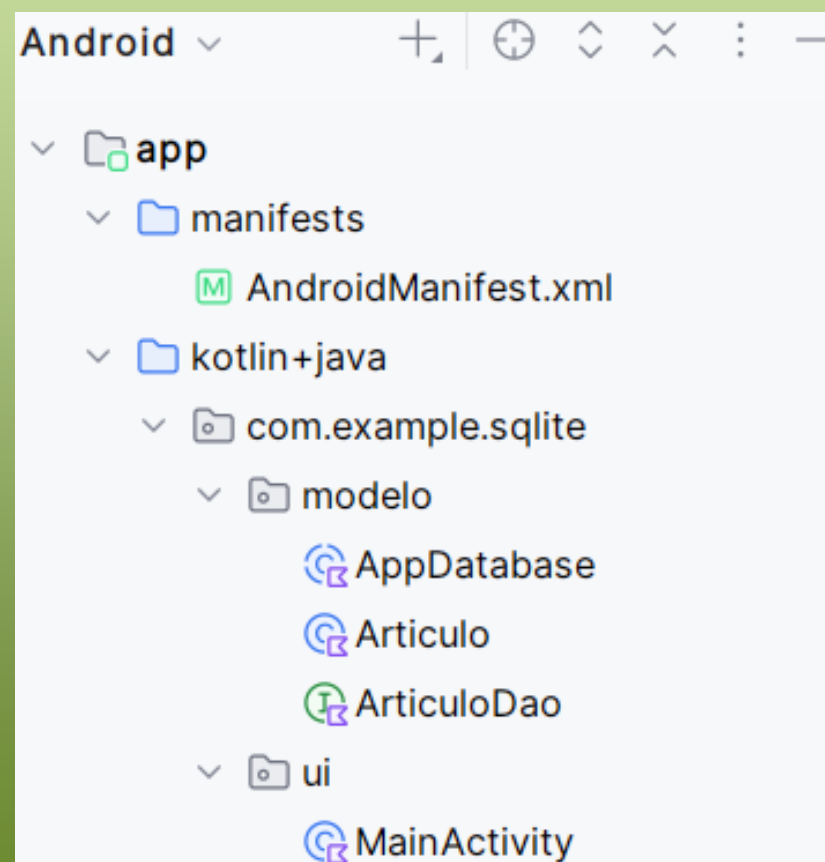
- Dependencias:

```
//Librería que contiene el núcleo de Room (clases Room y RoomDatabase)
implementation("androidx.room:room-runtime:2.8.4")
//En combinación con el plugin kotlin("kapt"), permite compilar e implementar la BD y el DAO
kapt("androidx.room:room-compiler:2.8.4")
//Librería que integra las Corrutinas con Room, permitiendo usar suspend en los DAOs
implementation("androidx.room:room-ktx:2.8.4")
//Librería para utilizar Corrutinas en Kotlin
implementation("org.jetbrains.kotlinx:kotlinx-coroutines-android:1.7.3")
```

2.- SQLite con Room

Organización del proyecto (MVC)

- Para estructurar el proyecto utilizaremos el patrón de diseño *Modelo-Vista-Controlador*.



2.- SQLite con Room

Entidad

- Creamos la data class que representa la tabla de la BD. Cada objeto de esta clase será una fila de la tabla.
- Con la anotación `@Entity` indicamos el nombre de la tabla. Con `@PrimaryKey`, la clave primaria.

```
Articulo.kt x
1 package com.example.sqlite.modelo
2
3 import androidx.room.Entity
4 import androidx.room.PrimaryKey
5
6 @Entity(tableName = "articulos")
7 data class Articulo(
8     @PrimaryKey
9     val codigo: Int,
10    val descripcion: String,
11    val stock: Int,
12    val precio: Float
13 )
```

2.- SQLite con Room

DAO (Data Access Object)

- El DAO es la interfaz que define las operaciones sobre la BD. Room se encarga de generar internamente la implementación. De esta manera evitamos escribir SQL directamente en Activities y Fragments, dejando el código más limpio.
- Para indicar que la interfaz es un DAO utilizamos la anotación *@Dao*. Las diferentes operaciones las declararemos con las anotaciones *@Insert*, *@Update*, *@Delete* y *@Query*.

2.- SQLite con Room

```

ArticuloDao.kt
1 package com.example.sqlite.modelo
2
3 import androidx.room.Dao
4 import androidx.room.Insert
5 import androidx.room.Query
6 import androidx.room.Update
7 import androidx.room.Delete
8
9 @Dao
10 interface ArticuloDao {
11     @Insert
12     suspend fun insertar(articulo: Articulo)
13
14     @Update
15     suspend fun modificar(articulo: Articulo)
16
17     @Delete
18     suspend fun eliminar(articulo: Articulo)
19
20     @Query(value = "SELECT * FROM articulos WHERE codigo = :codigo")
21     suspend fun buscarPorCodigo(codigo: Int): Articulo?
22
23     @Query(value = "SELECT * FROM articulos WHERE descripcion LIKE '%' || :descripcion || '%'"')
24     suspend fun buscarPorDescripcion(descripcion: String): List<Articulo>
25 }

```

2.- SQLite con Room

Base de Datos (clase AppDatabase)

```

AppDatabase.kt
1 package com.example.sqlite.modelo
2
3 import android.content.Context
4 import androidx.room.Database
5 import androidx.room.Room
6 import androidx.room.RoomDatabase
7
8 @Database(entities = [Articulo::class], version = 1)
9 @abstract class AppDatabase : RoomDatabase() {
10     abstract fun articuloDao(): ArticuloDao
11     companion object {
12         private var instance: AppDatabase? = null
13         fun getInstance(context: Context): AppDatabase {
14             if (instance == null) {
15                 instance = Room.databaseBuilder(
16                     context.applicationContext,
17                     AppDatabase::class.java,
18                     "tienda"
19                 ).build()
20             }
21             return instance as AppDatabase
22         }
23     }
24 }

```

2.- SQLite con Room

- La clase ***AppDatabase*** representa una base de datos Room. Se utiliza *abstract* para que no se puedan instanciar objetos de esta clase.
- Mediante la anotación *@Database* indicamos las tablas que tiene la BD en entities (en este caso solo una) y la versión de esta BD.
- El método ***articuloDao()*** devuelve una referencia del DAO con las operaciones que se pueden realizar sobre la BD. Se utiliza *abstract* para que no se pueda sobrescribir.
- El método ***getDatabase()*** devuelve una instancia única de la BD. Crea la instancia solo si no existe. Se usa *companion object* para que la BD sea única en toda la app (Singleton).

2.- SQLite con Room

Uso de Room en la Activity

- Para realizar cualquier operación sobre la BD hay que seguir los siguientes pasos:
 - Obtener la base de datos.
 - Lanzar la operación correspondiente del DAO en una corrutina.
- Hay que tener en cuenta que Room es capaz de ejecutar la consulta en el hilo de la corrutina y volver automáticamente al hilo principal (UIThread). No es necesario utilizar la función "runOnUiThread".

2.- SQLite con Room

Eliminar artículo

```
private fun insertarArticulo() {
    val codigo: Int? = editTextCodigo.text.toString().toIntOrNull()
    val descripcion: String = editTextDescripcion.text.toString()
    val stock: Int? = editTextStock.text.toString().toIntOrNull()
    val precio: Float? = editTextPrecio.text.toString().toFloatOrNull()

    if(codigo == null || descripcion == "" || stock == null || precio == null) {
        Toast.makeText(applicationContext, resId = R.string.no_datos_articulo_msg, duration = Toast.LENGTH_SHORT).show()
    } else {
        val articulo = Articulo(codigo, descripcion, stock, precio)
        lifecycleScope.launch {
            val db = AppDatabase.getDatabase(applicationContext)
            db.articuloDao().insertar(articulo)
            //Se muestra mensaje de datos guardados correctamente
            Toast.makeText(applicationContext, resId = R.string.articulo_guardado_ok_msg, duration = Toast.LENGTH_SHORT).show()
            //Se limpian los EditText
            editTextCodigo.setText("")
            editTextDescripcion.setText("")
            editTextStock.setText("")
            editTextPrecio.setText("")
        }
    }
}
```

Añadir artículo

```
private fun eliminarArticulo() {
    val codigo: Int? = editTextCodigo.text.toString().toIntOrNull()
    if(codigo == null) {
        Toast.makeText(applicationContext, resId = R.string.no_codigo_articulo_msg, duration = Toast.LENGTH_SHORT).show()
    } else {
        lifecycleScope.launch {
            val db = AppDatabase.getDatabase(applicationContext)
            val articulo = db.articuloDao().buscarPorCodigo(codigo)
            if (articulo != null) {
                db.articuloDao().eliminar(articulo)
                //Se muestra mensaje de artículo eliminado correctamente
                Toast.makeText(applicationContext, resId = R.string.articulo_borrado_ok_msg, duration = Toast.LENGTH_SHORT).show()
                //Se limpian los EditText
                editTextCodigo.setText("")
                editTextDescripcion.setText("")
                editTextStock.setText("")
                editTextPrecio.setText("")
            } else {
                Toast.makeText(applicationContext, resId = R.string.no_articulo_cod_msg, duration = Toast.LENGTH_SHORT).show()
            }
        }
    }
}
```

2.- SQLite con Room

Modificar artículo

```
private fun modificarArticulo() {  
    val codigo: Int? = editTextCodigo.text.toString().toIntOrNull()  
    val descripcion: String = editTextDescripcion.text.toString()  
    val stock: Int? = editTextStock.text.toString().toIntOrNull()  
    val precio: Float? = editTextCodigo.text.toString().toFloatOrNull()  
  
    if(codigo == null || descripcion == "" || stock == null || precio == null) {  
        Toast.makeText(applicationContext, R.string.no_datos_articulo_msg, duration = Toast.LENGTH_SHORT).show()  
    } else {  
        val articulo = Articulo(codigo, descripcion, stock, precio)  
        lifecycleScope.launch {  
            val db = AppDatabase.getDatabase(applicationContext)  
            db.articuloDao().modificar(articulo)  
            //Se muestra mensaje de datos modificados correctamente  
            Toast.makeText(applicationContext, R.string.articulo_modificado_ok_msg, duration = Toast.LENGTH_SHORT).show()  
            //Se limpian los EditText  
            editTextCodigo.setText("")  
            editTextDescripcion.setText("")  
            editTextStock.setText("")  
            editTextPrecio.setText("")  
        }  
    }  
}
```

2.- SQLite con Room

Consultar artículo por descripción

```
private fun consultaPorCodigo() {
    val codigo: Int? = editTextCodigo.text.toString().toIntOrNull()
    if(codigo == null) {
        Toast.makeText(applicationContext, resId = R.string.no_codigo_articulo_msg, duration = Toast.LENGTH_SHORT).show()
    } else {
        lifecycleScope.launch {
            val db = AppDatabase.getDatabase(applicationContext)
            val articulo = db.articuloDao().buscarPorCodigo(codigo)
            if (articulo != null) {
                //Se actualizan los EditText con los datos del articulo consultado
                editTextDescripcion.setText(articulo.descripcion)
                editTextStock.setText(articulo.stock.toString())
                editTextPrecio.setText(articulo.precio.toString())
            } else {
                Toast.makeText(applicationContext, resId = R.string.no_articulo_cod_msg, duration = Toast.LENGTH_SHORT).show()
            }
        }
    }
}
```

Consultar artículo por código

```
private fun consultaPorDescripcion() {
    val descripcion: String = editTextDescripcion.text.toString()
    if(descripcion == "") {
        Toast.makeText(applicationContext, resId = R.string.no_desc_articulo_msg, duration = Toast.LENGTH_SHORT).show()
    } else {
        lifecycleScope.launch {
            val db = AppDatabase.getDatabase(applicationContext)
            val listaArticulos: List<Articulo> = db.articuloDao().buscarPorDescripcion(descripcion)
            if (listaArticulos.isNotEmpty()) {
                //Se coge el primer articulo de la lista (el primero en el que la descripción buscada coincide)
                val articulo = listaArticulos[0]
                //Se actualizan los EditText con los datos del articulo seleccionado
                editTextCodigo.setText(articulo.codigo.toString())
                editTextStock.setText(articulo.stock.toString())
                editTextPrecio.setText(articulo.precio.toString())
            } else {
                Toast.makeText(applicationContext, resId = R.string.no_articulo_desc_msg, duration = Toast.LENGTH_SHORT).show()
            }
        }
    }
}
```

EJERCICIOS

- **Ejercicio 02:** Modifica el ejemplo anterior para añadir una funcionalidad que permita consultar todos los artículos existentes en la base de datos y mostrarlos en un RecyclerView.
- Para ello, necesitarás añadir el botón "Consultar todos" en la actividad principal de la app y crear otra actividad en la que muestres el listado de artículos:

EJERCICIOS

CÓDIGO DEL ARTÍCULO

DESCRIPCIÓN DEL ARTÍCULO

STOCK

PRECIO

ALTA BAJA MODIFICACIÓN

CONSULTA POR CÓDIGO CONSULTA POR DESCRIPCIÓN

CONSULTAR TODOS

Código:	1
Descripción:	Memoria RAM
Stock:	25
Precio:	30.0 €

Código:	2
Descripción:	Monitor
Stock:	5
Precio:	290.0 €

Código:	3
Descripción:	CPU
Stock:	20
Precio:	550.0 €

Código:	4
Descripción:	Raton
Stock:	15
Precio:	24.0 €

Código:	5
Descripción:	Impresora
Stock:	80
Precio:	199.0 €

PMDM

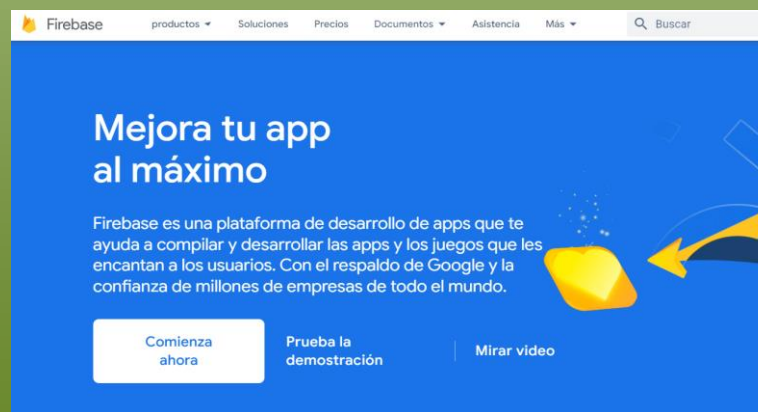
ALMACENAMIENTO PERMANENTE DE
DATOS



4. Conexión Firebase

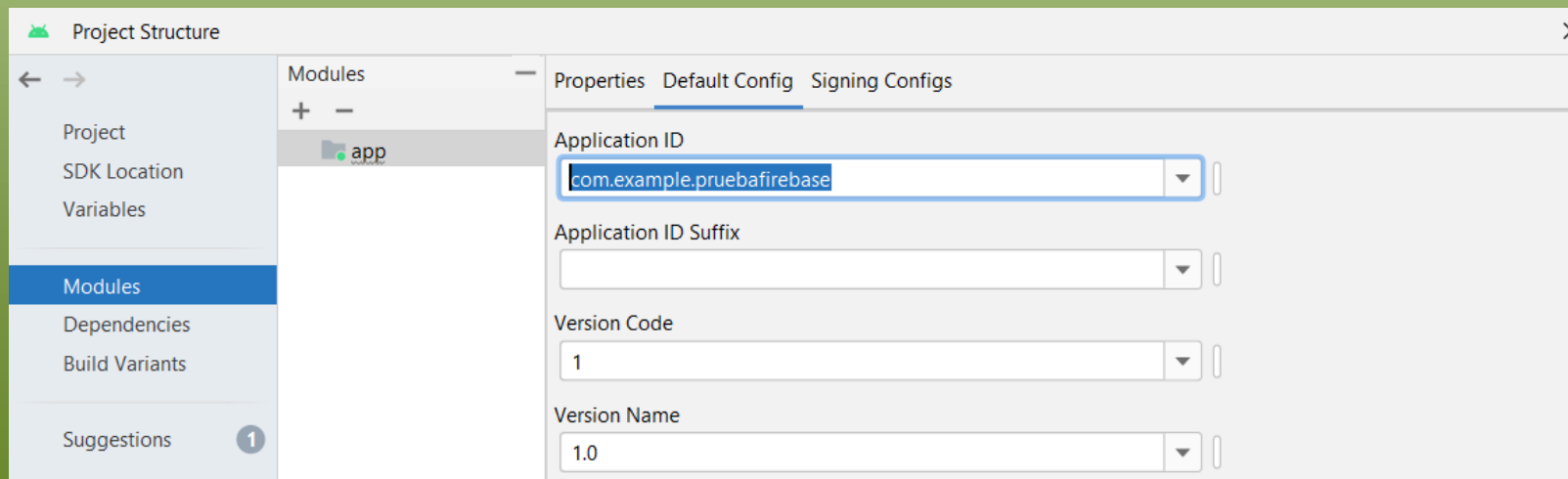
4.- Conexión Firebase

- Firebase es una plataforma para el desarrollo de aplicaciones móviles que facilita la conexión de nuestras aplicaciones con una base de datos en tiempo real, almacenando nuestros datos y permitiendo su sincronización. Sirve para aplicaciones Android, IOS y Web.
- En mayo del 2016 Google la presentó ofreciendo multitud de funcionalidades: autenticación, almacenamiento, alojamiento, notificaciones WebApps y mensajería en la nube.



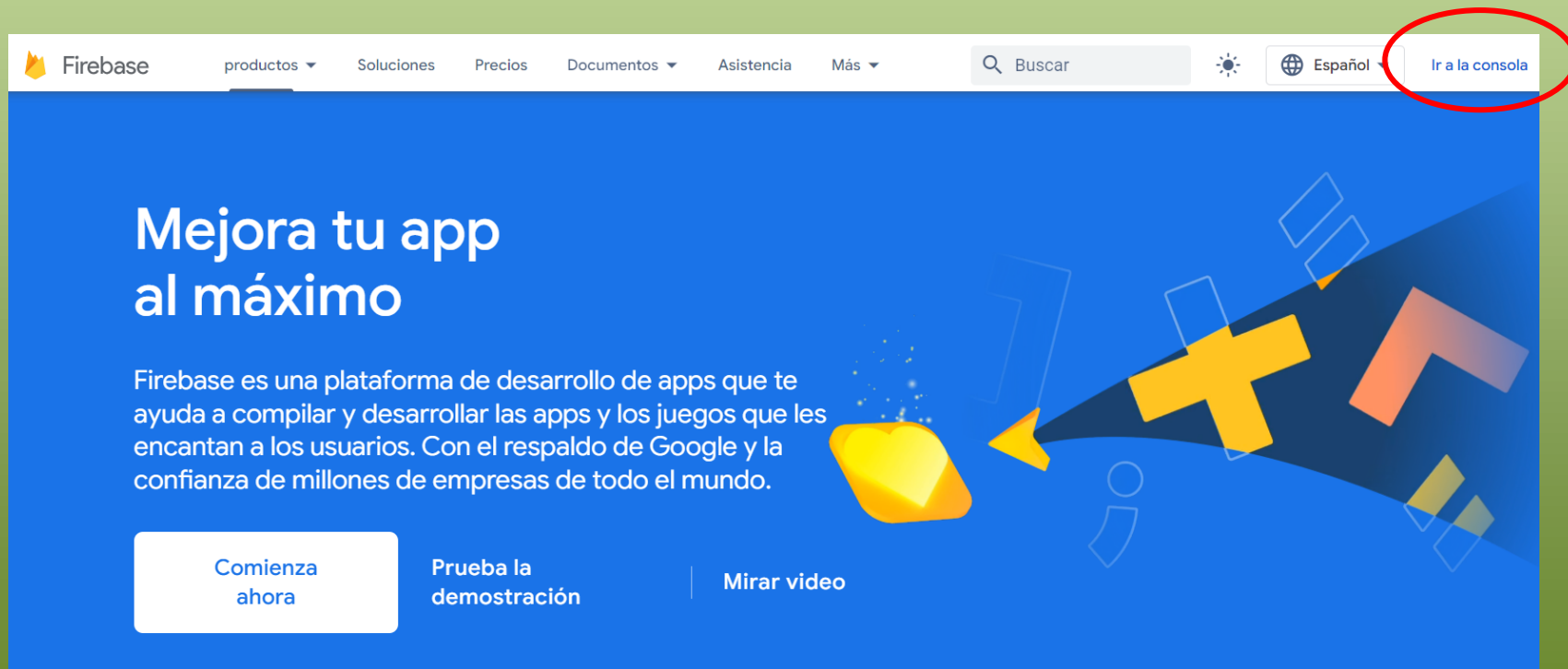
4.- Conexión Firebase

- Vamos a comenzar a utilizarla creando un proyecto en Android Studio llamado, por ejemplo, *PruebaFirebase*.
- Una vez creado el proyecto, vamos a File -> Project Structure -> Modules -> app -> Default Config y copiamos el **Application ID** de nuestra app.



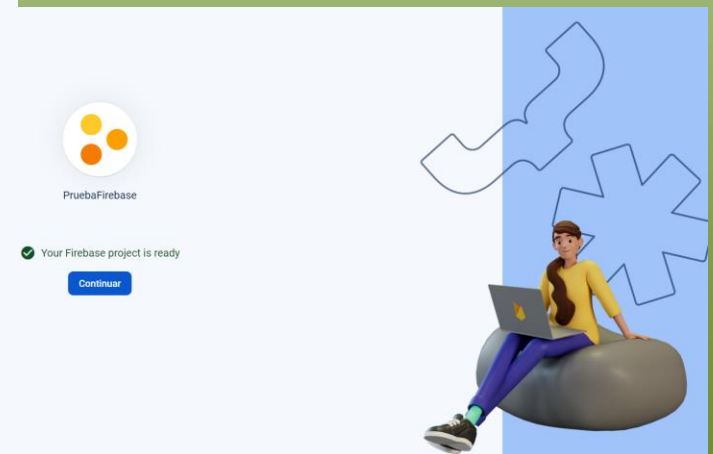
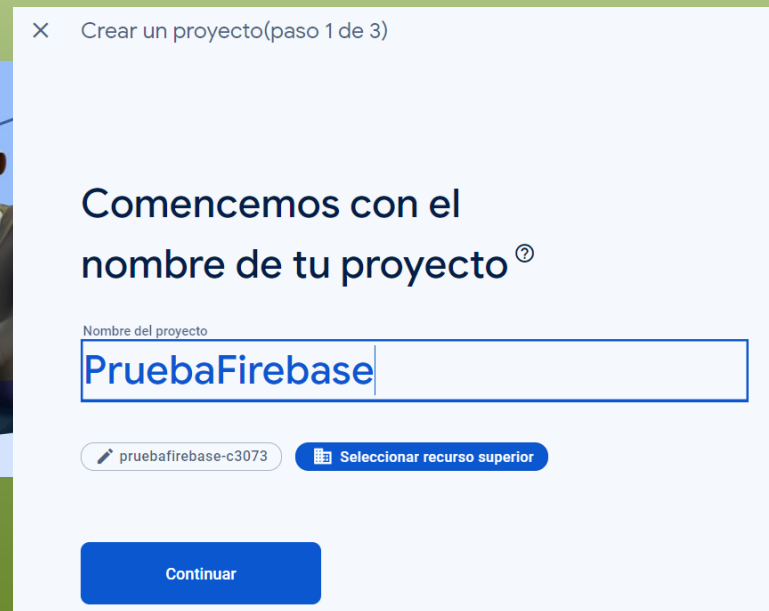
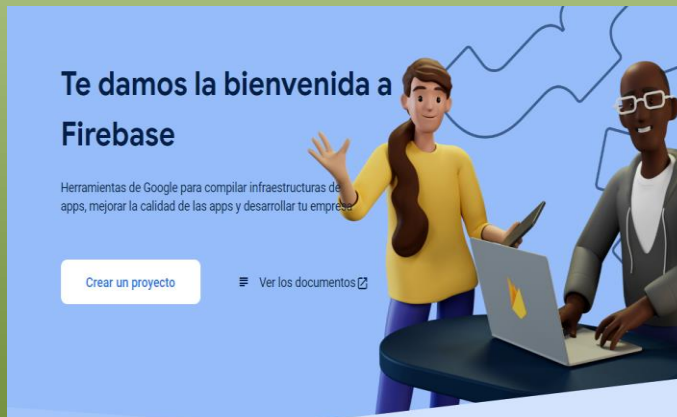
4.- Conexión Firebase

- A continuación visitamos la Web de Firebase y, desde la página de bienvenida, accedemos a la consola:



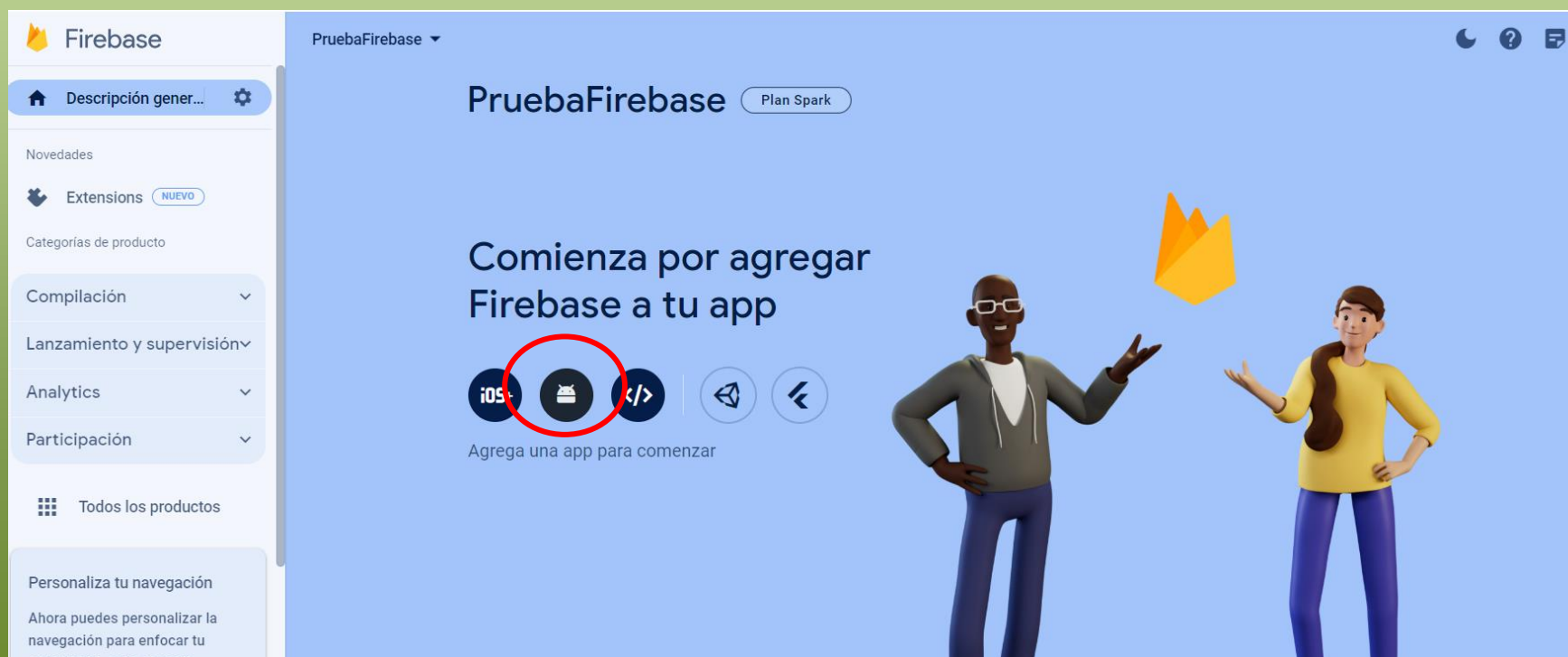
4.- Conexión Firebase

- Una vez en la consola, pulsamos en **Crear un proyecto**. Escribimos el nombre de nuestro proyecto, aceptamos los términos y condiciones, elegimos nuestro país y creamos nuestro proyecto.



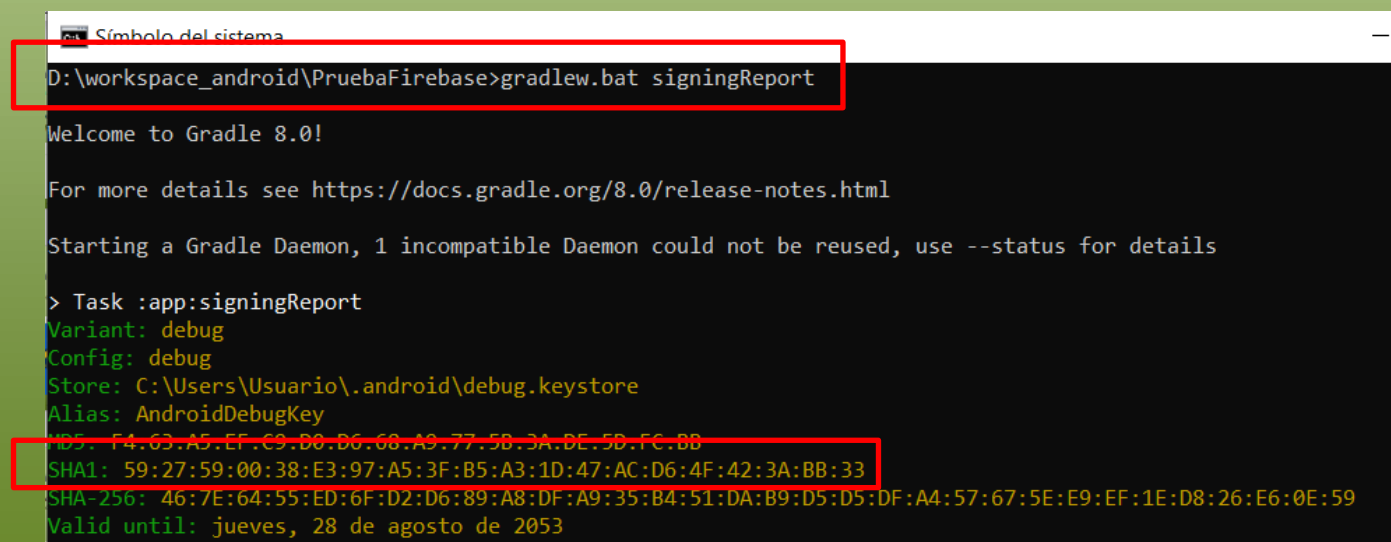
4.- Conexión Firebase

- Para terminar, seleccionamos **Agrega una app para comenzar** pulsando en el icono de Android:



4.- Conexión Firebase

- Pegamos nuestro *Application ID* en **Nombre del paquete Android** y el **Certificado de firma SHA-1 de depuración** para nuestra app. Posteriormente pulsamos en **Registrar app**.
- Para obtener el SHA-1 necesitamos ejecutar el comando **gradlew.bat signingReport** de Gradle, desde la ubicación de nuestro proyecto:



```

D:\workspace_android\PruebaFirebase>gradlew.bat signingReport

Welcome to Gradle 8.0!

For more details see https://docs.gradle.org/8.0/release-notes.html

Starting a Gradle Daemon, 1 incompatible Daemon could not be reused, use --status for details

> Task :app:signingReport
Variant: debug
Config: debug
Store: C:\Users\Usuario\.android\debug.keystore
Alias: AndroidDebugKey
MD5: F4:63:A5:EF:C9:D0:D6:68:A9:77:5B:3A:DE:5B:FC:BB
SHA1: 59:27:59:00:38:E3:97:A5:3F:B5:A3:1D:47:AC:D6:4F:42:3A:BB:33
SHA-256: 46:7E:64:55:ED:6F:D2:D6:89:A8:DF:A9:35:B4:51:DA:B9:D5:D5:DF:A4:57:67:5E:E9:EF:1E:D8:26:E6:0E:59
Valid until: jueves, 28 de agosto de 2053
  
```

4.- Conexión Firebase

×

Agrega Firebase a tu app para Android

1

Registrar app

Nombre del paquete de Android ?

com.example.pruebafirebase

Sobrenombre de la app (opcional) ?

Mi app para Android

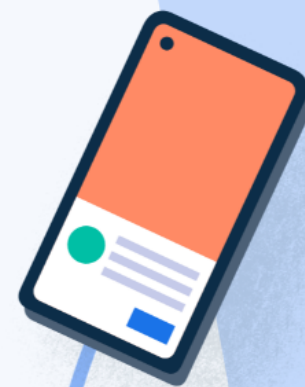
Certificado de firma SHA-1 de depuración (opcional) ?

59:27:59:00:38:E3:97:A5:3F:B5:A3:1D:47:AC:D6:4F:42

1

Obligatoria para Dynamic Links y el Acceso con Google o la asistencia con un número de teléfono en Auth. Puedes editar las claves SHA-1 en Configuración.

Registrar app



Profesor: Óscar Laguna García
Última versión: Daniel Fernández Rodríguez

PROGRAMACIÓN MULTIMEDIA Y DISPOSITIVOS MÓVILES

40

- El siguiente paso es descargar el archivo **google-services.json** y colocarlo en nuestro proyecto como se indica en la web de Firebase. Pulsamos en **Siguiente**.

4.- Conexión Firebase

×

Agrega Firebase a tu app para Android

✓

Registrar app

Nombre del paquete de Android: com.example.pruebafirebase

2

Descargar y, luego, agregar el archivo de configuración

Android Studio a continuación | [Unity](#) [C++](#)

↓

Descargar google-services.json

Cambia a la vista **Proyecto** en Android Studio para ver el directorio raíz de tu proyecto.

Mueve el archivo `google-services.json` descargado al directorio raíz de tu módulo (nivel de app).



google-services.json

Project

MyApplication [My Application]

.gradle

.idea

app

libs

src

.gitignore

build.gradle.kts

google-services.json

proguard-rules.pro

gradle

Siguiente

Profesor: Óscar Laguna García
Última versión: Daniel Fernández Rodríguez

PROGRAMACIÓN MULTIMEDIA Y DISPOSITIVOS MÓVILES

41

4.- Conexión Firebase

- El último paso consiste en añadir un par de líneas en el **gradle del proyecto** y en el **gradle de la aplicación**, y después sincronizar.

1. Para que los SDK de Firebase puedan acceder a los valores de configuración de `google-services.json`, necesitas el complemento Gradle de los servicios de Google.

☒ DSL de Kotlin (build.gradle.kts) ☐ Groovy (build.gradle)

Agrega el complemento como una dependencia a tu archivo `build.gradle.kts` de nivel de proyecto:

Archivo de Gradle de nivel de raíz (nivel de proyecto) (<project>/build.gradle.kts):

```
plugins {
    // ...

    // Add the dependency for the Google services Gradle plugin
    id("com.google.gms.google-services") version "4.4.0" apply false
}
```

2. Luego, en el archivo `build.gradle.kts` del módulo (nivel de app), agrega los complementos `google-services` y cualquier SDK de Firebase que quieras usar en tu app:

Archivo de Gradle del módulo (nivel de app) (<project>/<app-module>/build.gradle.kts):

```
plugins {
    id("com.android.application")
    // Add the Google services Gradle plugin
    id("com.google.gms.google-services")
}
```



4.- Conexión Firebase

- A continuación vamos a añadir las siguientes bibliotecas de la SDK de Firebase al build.gradle.kts del módulo app de nuestro proyecto, para empezar a crear nuestra base de datos:
 - **BoM:** *Se encarga de descargar las últimas versiones compatibles existentes en la SDK de Firebase para las aplicaciones Android.*
 - **Realtime Database:** *Biblioteca para usar la base de datos de Firebase.*
- Sincronizamos el gradle para descargar las dependencias.

Firebase	productos	Soluciones	Precios	Docs	Comunidad	Ayuda	Buscar
	Filtrar						
Aspectos básicos		Visualización de In-App Messaging			com.google.firebase:firebase-inappmessaging-display-ktx	20.3.5	
		Instalaciones de Firebase			com.google.firebase:firebase-installations-ktx	17.1.4	
		API de Firebase ML Model Downloader			com.google.firebase:firebase-ml-modeldownloader-ktx	24.1.3	
		Performance Monitoring			com.google.firebase:firebase-perf-ktx	20.4.1	
		Complemento de Performance Monitoring			com.google.firebase:perf-plugin	1.4.2	
Comience con Firebase		Realtime Database			com.google.firebase:firebase-database-ktx	20.2.2	
Agregar Firebase a una aplicación		Remote Config			com.google.firebase:firebase-config-ktx	21.4.1	
Plataformas Apple (iOS+)							
Android							
Web							
Aleto							

```

dependencies {
    implementation("androidx.core:core-ktx:1.9.0")
    implementation("androidx.appcompat:appcompat:1.6.1")
    implementation("com.google.android.material:material:1.11.0")
    implementation("androidx.constraintlayout:constraintlayout:2.1.4")
    testImplementation("junit:junit:4.13.2")
    androidTestImplementation("androidx.test.ext:junit:1.1.5")
    androidTestImplementation("androidx.test.espresso:espresso-core:3.5.1")

    //Firebase

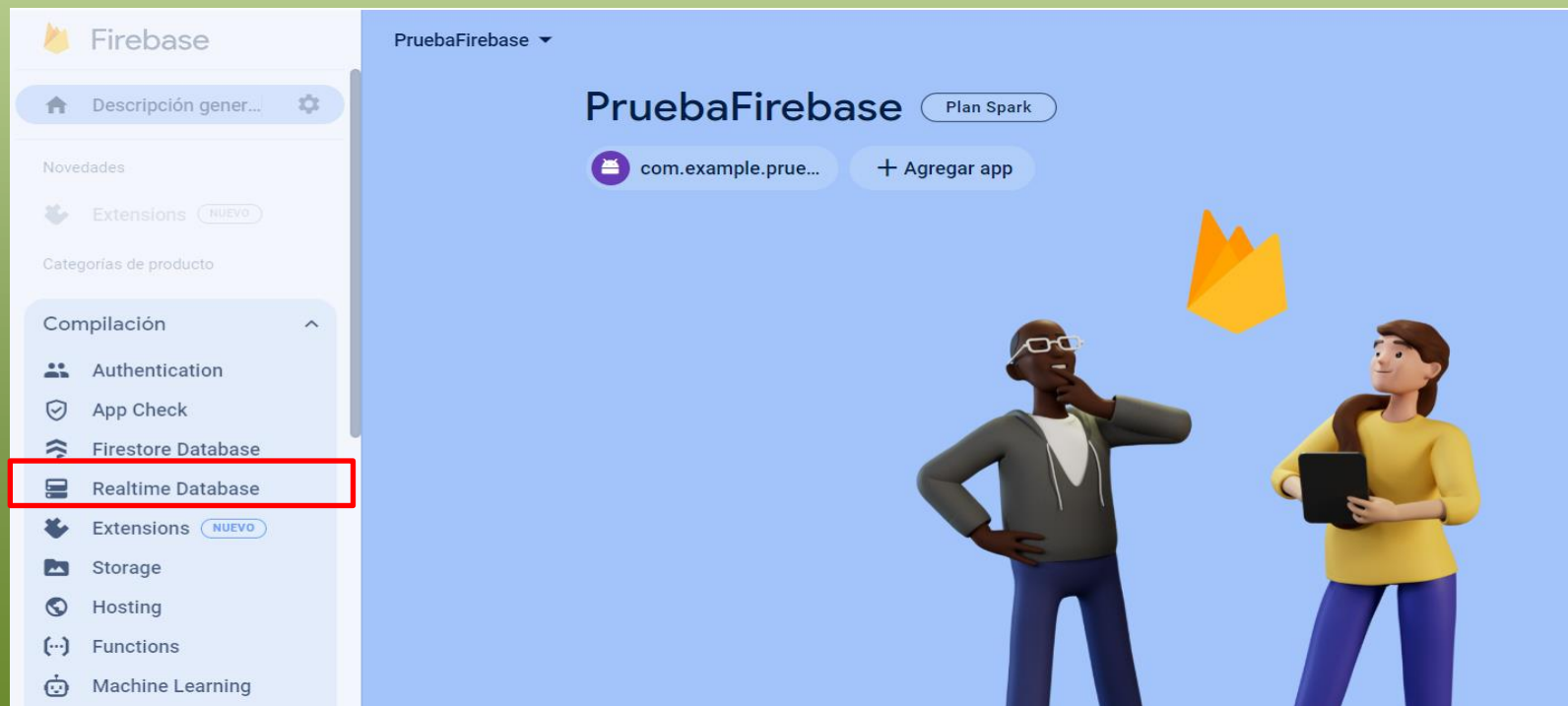
    //BoM: Se encarga de descargar las últimas versiones compatibles existentes en la SDK
    //de Firebase para las aplicaciones Android
    implementation(platform("com.google.firebase:firebase-bom:32.7.1"))

    //Realtime Database
    implementation("com.google.firebase:firebase-database-ktx")
  }

```

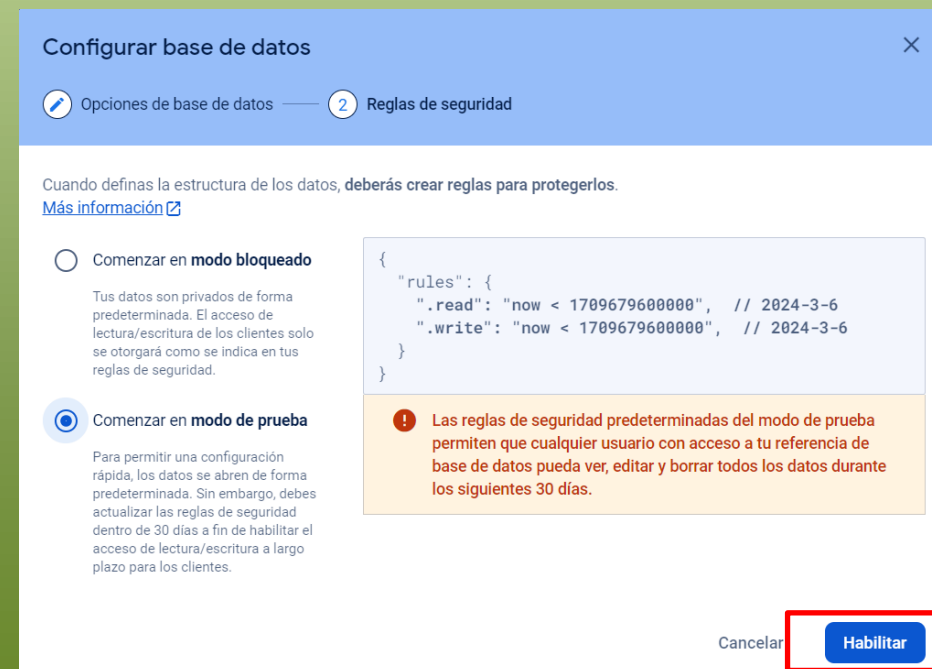
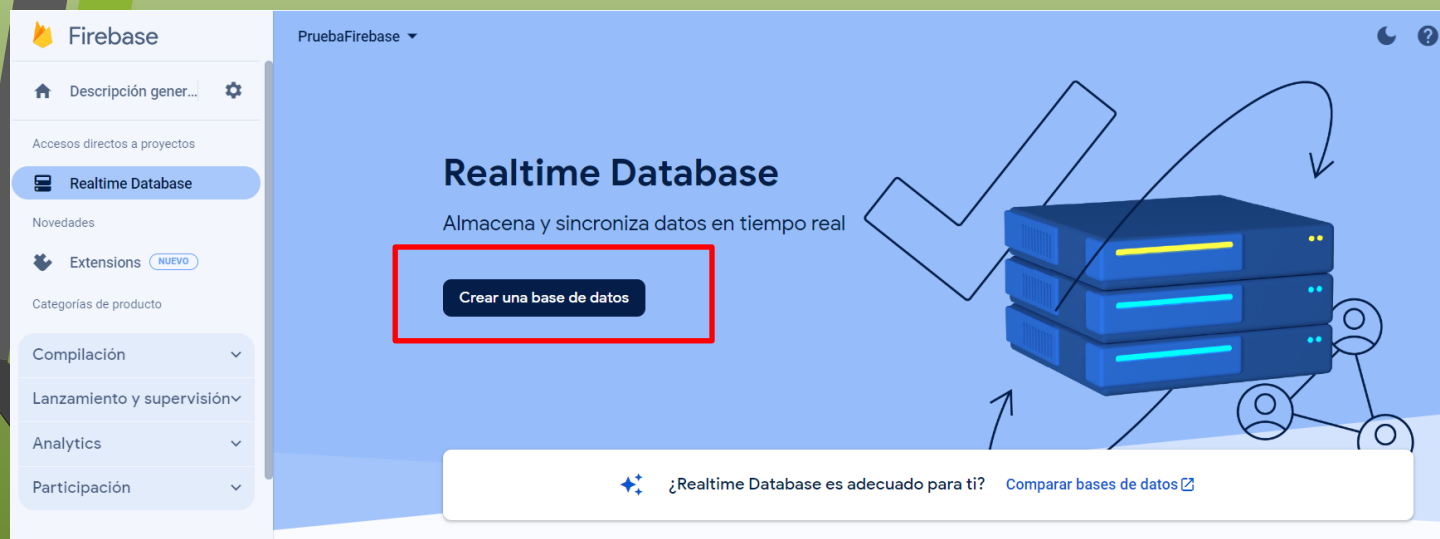
4.- Conexión Firebase

- Vamos a crear una Realtime Database en FireBase. Para ello, seleccionamos nuestro proyecto "PruebaFirebase" y elegimos la opción **Realtime Database** en el menú **Compilación** de la parte izquierda de la ventana:



4.- Conexión Firebase

- Pulsamos en **Crear una base de datos** y seleccionamos **Comenzar en modo prueba**, para configurar reglas de escritura y lectura que no necesiten autenticación (ya que aún no la hemos implementado):



4.- Conexión Firebase

- Accedemos a la pestaña **Reglas** de Realtime Database y establecemos las reglas de lectura y escritura a "true" (acceso público), para poder acceder sin autenticación desde nuestra app Android:

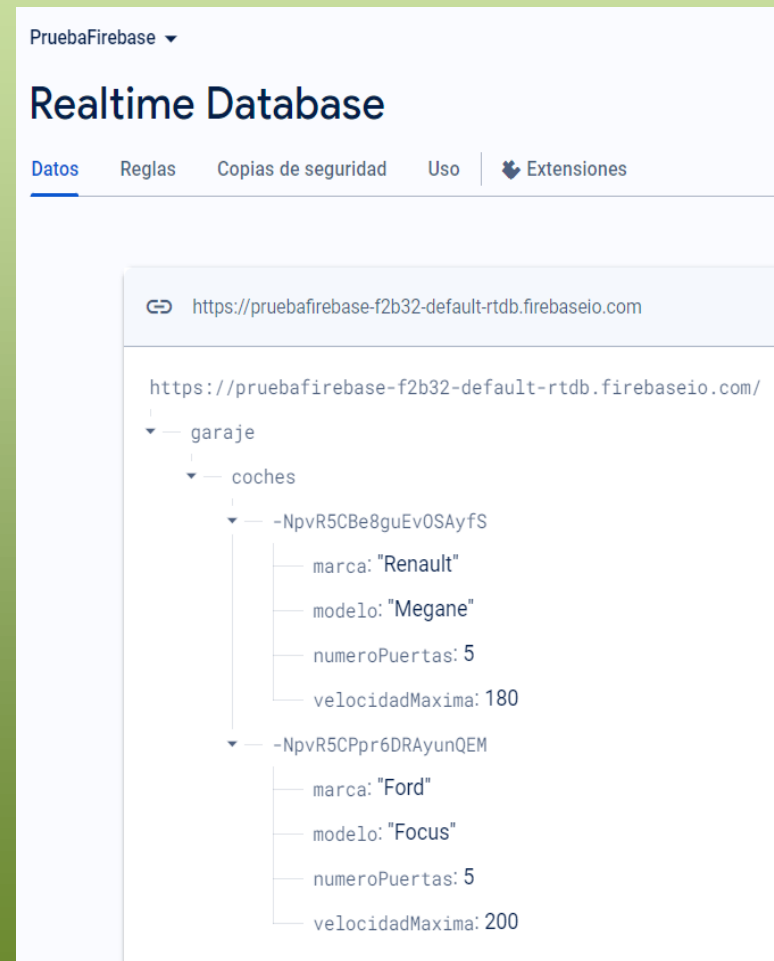


4.- Conexión Firebase

- La bases de datos en Firebase son NO RELACIONALES, Y NO PERMITEN CONSULTAS SQL, ya que almacenan la información en un sencillo fichero **JSON**. Además, Firebase en su versión gratuita, solo permiten 100 conexiones simultáneas.
- Sabiendo esto, vamos a ver cómo escribiríamos en la base de datos Firebase desde nuestra aplicación Android:

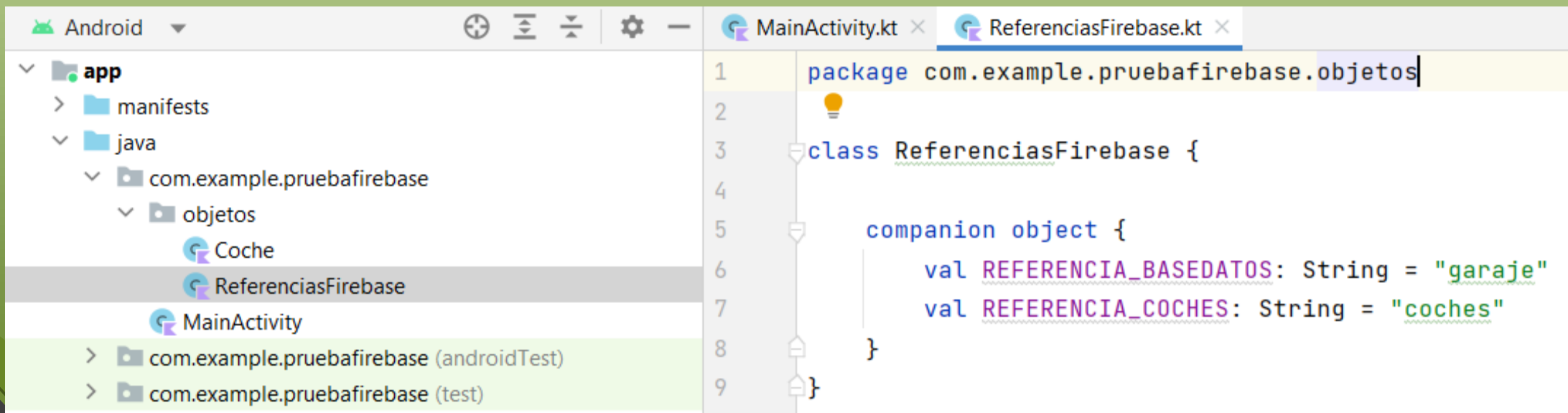
4.- Conexión Firebase

- La captura muestra el JSON resultado que queremos obtener en Firebase, una vez escritos los datos desde nuestra app.
- Por buscar un símil a las bases de datos SQL, **"garaje"** sería la base de datos, **"coches"** sería la tabla, el identificador de objeto (key) sería la clave, y los atributos del objeto (marca, modelo, numeroPuertas y velocidadMaxima) serían los campos de la tabla.



4.- Conexión Firebase

- Comenzaremos creando una clase donde guardaremos las referencias a la base de datos Firebase. Esto es, el nombre de la base de datos y el nombre de las tablas:



The screenshot shows the Android Studio IDE. On the left, the project structure is visible under the 'app' folder, showing the 'java' directory with the package 'com.example.pruebafirebase' and the class 'ReferenciasFirebase'. On the right, the code for 'ReferenciasFirebase.kt' is displayed. The code defines a package, a class, and a companion object with two static references to Firebase databases.

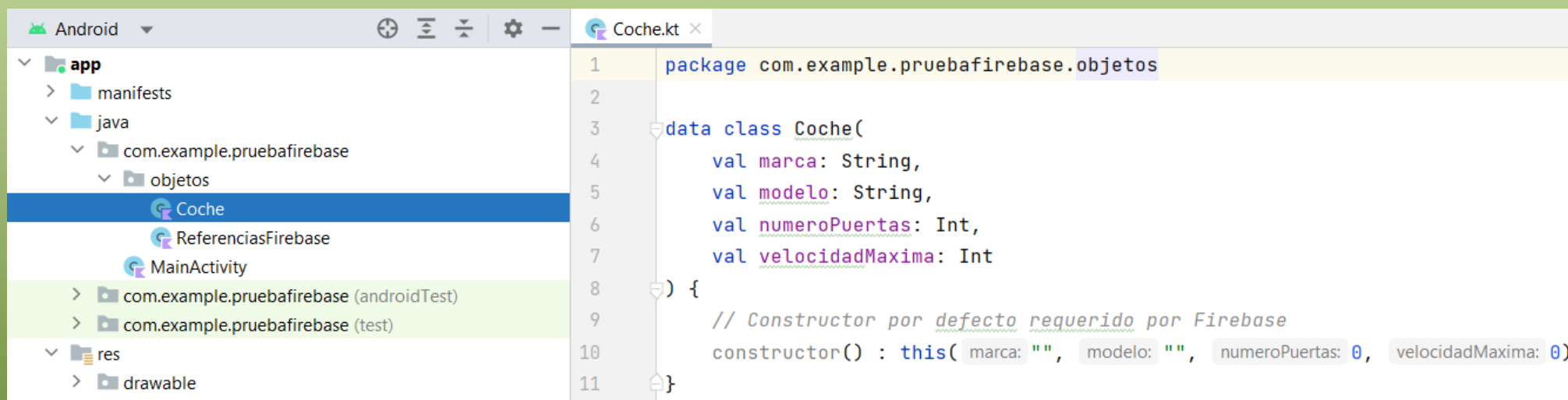
```

1 package com.example.pruebafirebase.objetos
2
3 class ReferenciasFirebase {
4
5     companion object {
6         val REFERENCIA_BASEDATOS: String = "garaje"
7         val REFERENCIA_COCHES: String = "coches"
8     }
9 }

```

4.- Conexión Firebase

- También necesitaremos una clase para instanciar los objetos que luego volcaremos a la base de datos. Para ello, implementamos una **data class** con el nombre **Coche**:



The screenshot shows the Android Studio interface. On the left, the 'Project' view displays the file structure: 'app' > 'java' > 'com.example.pruebafirebase' > 'objetos'. The 'Coche.kt' file is selected and open in the editor. The code defines a data class 'Coche' with four properties: 'marca' (String), 'modelo' (String), 'numeroPuertas' (Int), and 'velocidadMaxima' (Int). A default constructor is implemented, initializing all properties with empty strings or zero. The package is 'com.example.pruebafirebase.objetos'.

```
1 package com.example.pruebafirebase.objetos
2
3 data class Coche(
4     val marca: String,
5     val modelo: String,
6     val numeroPuertas: Int,
7     val velocidadMaxima: Int
8 ) {
9     // Constructor por defecto requerido por Firebase
10    constructor() : this( marca: "", modelo: "", numeroPuertas: 0, velocidadMaxima: 0)
11 }
```

- NOTA:*** Para que las librerías de Firebase puedan mapear los datos que leen de la base de datos en nuestra data class, se debe implementar el constructor por defecto sin argumentos de la clase.

4.- Conexión Firebase

- Implementamos el código en el método onCreate() de la MainActivity.kt y, al ejecutar nuestra aplicación, veremos como en la base de datos de Firebase se crean los objetos indicados en nuestra app:

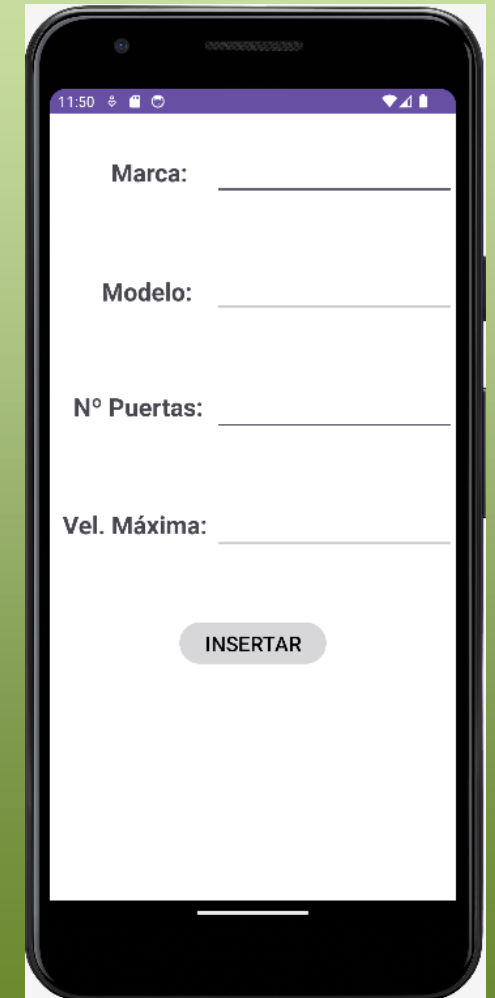
```

1 package com.example.pruebafirebase
2
3 import ...
4
5
6
7
8
9
10 class MainActivity : AppCompatActivity() {
11
12     override fun onCreate(savedInstanceState: Bundle?) {
13         super.onCreate(savedInstanceState)
14         setContentView(R.layout.activity_main)
15
16         //Creamos una instancia de la base de datos
17         val firebaseDatabase: FirebaseDatabase = FirebaseDatabase.getInstance()
18
19         //Establecemos la referencia a la base de datos
20         val databaseReference: DatabaseReference = firebaseDatabase.getReference(ReferenciasFirebase.REFERENCIA_BASEDATOS)
21
22         //Creamos dos objetos "Coche"
23         val coche1: Coche = Coche( marca: "Renault", modelo: "Megane", numeroPuertas: 5, velocidadMaxima: 180)
24         val coche2: Coche = Coche( marca: "Ford", modelo: "Focus", numeroPuertas: 5, velocidadMaxima: 200)
25
26         //Insertamos ambos objetos en la bbdd, obteniendo la referencia a la "tabla" coches
27         databaseReference.child(ReferenciasFirebase.REFERENCIA_COCHES).push().setValue(coche1)
28         databaseReference.child(ReferenciasFirebase.REFERENCIA_COCHES).push().setValue(coche2)
29
30     }
31 }

```

4.- Conexión Firebase

- A continuación vamos a mejorar nuestra aplicación, de tal forma que el usuario introduzca los datos en los editText y que, al pulsar en el botón "INSERTAR", almacene los nuevos datos en Firebase, limpie los editText y muestre un Toast indicando que la operación de inserción se ha realizado correctamente.



4.- Conexión Firebase

- Para terminar, vamos a añadir un textView que se rellene con los datos de la tabla "coches" de Firebase.
- Además, implementaremos un **addValueEventListener** que esté siempre escuchando. De esta manera, cada vez que haya un cambio en la base de datos de Firebase, se nos actualizará el textView aplicando los nuevos cambios.



4.- Conexión Firebase

- Creamos una variable en la clase MainActivity que implemente el **addValueEventListener**.
- En el método **onDataChange** de este evento, realizamos un foreach que recorra todos los coches de la base de datos (se guardan en el objeto dataSnapshot) y mostramos los datos en el textView.

```
//Listener que escucha los cambios que se producen en la bbdd
private val valueEventListener = object : ValueEventListener {

    override fun onDataChange(dataSnapshot: DataSnapshot) {
        //Variable donde almacenamos el texto con los datos de todos los coches
        var datosCoches: String = ""
        //Se recorre el listado de coches obteniendo todos sus datos
        for (cocheSnapshot in dataSnapshot.children) {
            val cocheAux: Coche? = cocheSnapshot.getValue(Coche::class.java)
            if(cocheAux != null) {
                datosCoches = datosCoches + cocheAux.marca + " " + cocheAux.modelo +
                    " - Puertas: " + cocheAux.numeroPuertas + " - V.Máx: " + cocheAux.velocidadMaxima + "\n"
            }
        }
        //Se actualiza el textView con los datos de todos los coches
        textViewMostrar.setText(datosCoches)
    }

    override fun onCancelled(databaseError: DatabaseError) {
        //Error de acceso a la base de datos
        Toast.makeText(applicationContext, "Error de acceso", Toast.LENGTH_SHORT).show()
    }
}
```

4.- Conexión Firebase

- En el método onCreate() de la MainActivity.kt, utilizamos la referencia a la base de datos para establecer el addValueEventListener a la tabla "coches":

```
//Creamos una instancia de la base de datos
firebaseDatabase = FirebaseDatabase.getInstance()
//Establecemos la referencia a la base de datos
databaseReference = firebaseDatabase.getReference(ReferenciasFirebase.REFERENCIA_BASEDATOS)

//Asignamos el valueEventListener a la "tabla" coches de la base de datos
databaseReference.child(ReferenciasFirebase.REFERENCIA_COCHES).addValueEventListener(valueEventListener)
```

4.- Conexión Firebase

- Si quisiéramos presentar los datos de una manera más elegante, lo más adecuado sería mostrarlos en un RecyclerView.